

Lesson 13 Notes: Menu-Driven Matrix Program

1. Main idea of Lesson 13

Lesson 13 does not introduce a new matrix type.

Before this lesson, we learned many special matrices and their formulas. In Lesson 13, we learn how to put the matrix operations inside a menu-driven program.

A menu-driven program repeatedly shows choices to the user.

Example menu:

```
1. Create
2. Get
3. Set
4. Display
0. Exit
```

The user enters a choice, and the program performs that action.

So Lesson 13 is mainly about organizing matrix operations into an interactive program.

The main operations are:

- Create a matrix
- Get an element
- Set an element
- Display the full matrix
- Exit the program

2. Current progress before Lesson 13

Before Lesson 13, the course already covered the main storage ideas for special matrices.

Lessons 1 to 4: Diagonal matrix

A diagonal matrix stores values only on the main diagonal.

Condition:

```
i == j
```

Index formula:

```
index = i - 1
```

Example matrix:

```
5 0 0  
0 8 0  
0 0 9
```

Stored array:

```
A = [5, 8, 9]
```

Only the diagonal values are stored.

The zeros are structural zeros.

Lessons 5 to 8: Lower triangular matrix

A lower triangular matrix stores values on and below the main diagonal.

Condition:

```
i >= j
```

Row-major index formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

Example matrix:

```
1 0 0  
2 3 0  
4 5 6
```

Stored row-major array:

```
A = [1, 2, 3, 4, 5, 6]
```

Lesson 9: Upper triangular matrix

An upper triangular matrix stores values on and above the main diagonal.

Condition:

```
i <= j
```

The values below the main diagonal are structural zeros.

Lesson 10: Symmetric matrix

A symmetric matrix follows this rule:

```
M[i][j] == M[j][i]
```

That means the upper half and lower half mirror each other.

So we can store one triangular half and use the mirror property to get the other half.

Lesson 11: Tridiagonal and band matrices

A tridiagonal matrix stores only three diagonals:

1. The lower diagonal
2. The main diagonal
3. The upper diagonal

The important test is:

```
i - j
```

For a tridiagonal matrix:

```
i - j == 1    lower diagonal
i - j == 0    main diagonal
i - j == -1   upper diagonal
```

Everything else is treated as zero.

Lesson 12: Toeplitz matrix

A Toeplitz matrix has repeated values along each top-left to bottom-right diagonal.

Rule:

```
M[i][j] == M[i - 1][j - 1]
```

Instead of storing the full matrix, we store:

```
first row + first column without repeating the top-left value
```

For an `N x N` Toeplitz matrix, storage is:

```
2N - 1
```

3. Why Lesson 13 is important

Earlier lessons focused on formulas.

For each matrix type, we asked:

```
Which positions are stored?
What is the index formula?
How much memory is needed?
How do get, set, and display work?
```

Lesson 13 changes the focus.

Now we ask:

How do we let the user choose what operation to perform?

The matrix logic stays the same, but the program becomes interactive.

Instead of hard-coding calls like this:

```
set(...);  
get(...);  
display(...);
```

we let the user choose from a menu.

4. What is a menu-driven program?

A menu-driven program is a program that shows choices to the user again and again.

The user enters a number.

The program checks that number and runs the matching operation.

Example:

```
1. Create diagonal matrix  
2. Get element  
3. Set element  
4. Display matrix  
0. Exit
```

If the user enters , the program creates or fills the matrix.

If the user enters , the program gets an element.

If the user enters , the program sets an element.

If the user enters , the program displays the matrix.

If the user enters , the program stops.

5. Why Lesson 13 starts with a diagonal matrix

Lesson 13 starts with a diagonal matrix because it is the simplest special matrix.

For a diagonal matrix, the rule is:

```
i == j
```

This means a position is stored only when the row number and column number are the same.

Example stored positions:

```
M[1][1]  
M[2][2]  
M[3][3]
```

Non-stored positions:

```
M[1][2]  
M[1][3]  
M[2][1]  
M[2][3]  
M[3][1]  
M[3][2]
```

These non-diagonal positions are structural zeros.

So for a diagonal matrix:

```
if (i == j)  
    use A[i - 1];  
else  
    use 0;
```

6. Structural zeros reminder

A structural zero is a zero that exists because of the matrix rule.

It is not stored in memory.

For a diagonal matrix:

All positions where $i \neq j$ are structural zeros.

Example:

$M[1][3] = 0$

Why?

Because:

$1 \neq 3$

So $M[1][3]$ is not on the main diagonal.

The program does not need to store this zero.

It simply returns or prints 0 when needed.

7. Storage for the diagonal menu program

Suppose the user enters:

$n = 3$

This means the visible matrix is:

3×3

A full 3×3 matrix has:

$3 * 3 = 9$

visible positions.

But a diagonal matrix stores only:

```
n = 3
```

values.

So we create a 1D array of size `n`.

```
int *A;  
A = new int[n];
```

A safer beginner-friendly version is:

```
A = new int[n]{};
```

The `{}` initializes all values to zero.

That prevents garbage values before the user enters real values.

8. Matrix indexing and array indexing

In these lessons, matrix positions usually use 1-based indexing.

That means rows and columns start at `1`.

Examples:

```
M[1][1]  
M[2][2]  
M[3][3]
```

But C and C++ arrays use 0-based indexing.

That means array indexes start at `0`.

Examples:

```
A[0]  
A[1]  
A[2]
```


So we subtract `1` when converting a matrix position to an array index.

Mapping:

```
M[1][1] -> A[0]
M[2][2] -> A[1]
M[3][3] -> A[2]
```

Formula:

```
index = i - 1
```

This formula is used only when:

```
i == j
```

9. Main variables used in the program

A basic menu-driven diagonal matrix program uses these variables:

```
int n;
int *A;
int choice;
int i, j, x;
```

Meaning:

Variable	Meaning
<code>n</code>	Matrix dimension
<code>A</code>	1D array that stores diagonal values
<code>choice</code>	User menu choice
<code>i</code>	Row number
<code>j</code>	Column number
<code>x</code>	Value to store

Example:

```
int n;  
int *A;
```

If:

```
n = 3;
```

then the matrix behaves like a `3 x 3` matrix.

But `A` stores only the 3 diagonal values.

10. Basic menu loop structure

The menu is usually written using a `do-while` loop.

General structure:

```
do {  
    // show menu  
    // read user choice  
    // perform selected operation  
} while (choice != 0);
```

A `do-while` loop is useful because the menu should appear at least once.

The loop continues until the user enters `0`.

11. Why use switch-case?

A `switch` statement is useful when one variable can have many possible values.

Here, that variable is:

```
choice
```

Example:

```

switch (choice) {
    case 1:
        // create
        break;

    case 2:
        // get
        break;

    case 3:
        // set
        break;

    case 4:
        // display
        break;

    case 0:
        // exit
        break;

    default:
        // invalid choice
}

```

Each `case` handles one menu option.

The `break` statement stops the program from falling into the next case.

The `default` case handles invalid input, such as `9` or `-1`.

12. Full C++ menu-driven diagonal matrix program

```

#include <iostream>
using namespace std;

int main() {
    int n;
    int *A;
    int choice;
    int i, j, x;

    cout << "Enter matrix dimension: ";
    cin >> n;
}

```

```

// A diagonal matrix stores only n values.
A = new int[n]{};

do {
    cout << "\nMenu\n";
    cout << "1. Create diagonal matrix\n";
    cout << "2. Get element\n";
    cout << "3. Set element\n";
    cout << "4. Display matrix\n";
    cout << "0. Exit\n";
    cout << "Enter choice: ";
    cin >> choice;

    switch (choice) {
        case 1:
            cout << "Enter diagonal elements:\n";
            for (i = 1; i <= n; i++) {
                cin >> A[i - 1];
            }
            break;

        case 2:
            cout << "Enter row and column: ";
            cin >> i >> j;

            if (i == j) {
                cout << "Element is " << A[i - 1] << endl;
            } else {
                cout << "Element is 0" << endl;
            }
            break;

        case 3:
            cout << "Enter row, column, and new value: ";
            cin >> i >> j >> x;

            if (i == j) {
                A[i - 1] = x;
                cout << "Element updated.\n";
            } else {
                cout << "Cannot set non-diagonal position in a diagonal
matrix.\n";
            }
            break;

        case 4:
            for (i = 1; i <= n; i++) {

```

```

        for (j = 1; j <= n; j++) {
            if (i == j) {
                cout << A[i - 1] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
    break;

    case 0:
        cout << "Exiting program.\n";
        break;

    default:
        cout << "Invalid choice.\n";
    }

} while (choice != 0);

delete[] A;
return 0;
}

```

13. Explanation of the program

13.1 Header file

```
#include <iostream>
```

This is needed for input and output.

It lets us use:

```
cin
cout
```

13.2 Namespace

```
using namespace std;
```

This lets us write:

```
cout  
cin  
endl
```

instead of:

```
std::cout  
std::cin  
std::endl
```

13.3 Reading the matrix dimension

```
cout << "Enter matrix dimension: ";  
cin >> n;
```

The user enters the size of the square matrix.

If the user enters:

```
3
```

then the program creates a `3 x 3` diagonal matrix.

13.4 Allocating memory

```
A = new int[n]{};
```

This creates a dynamic array of size `n`.

For example, if:

```
n = 3;
```

then memory is created for:

```
A[0]  
A[1]  
A[2]
```

The `{}` initializes these values to zero.

13.5 Releasing memory

At the end of the program, we write:

```
delete[] A;
```

This releases the memory created using:

```
new int[n]{};
```

Important rule:

```
new[] must be matched with delete[]
```

Correct:

```
A = new int[n]{};  
delete[] A;
```

Wrong:

```
A = new int[n]{};  
delete A;
```

Use `delete[]` because `A` points to an array.

14. Menu option 1: Create

The create option fills the diagonal matrix.

Code:

```
case 1:
    cout << "Enter diagonal elements:\n";
    for (i = 1; i <= n; i++) {
        cin >> A[i - 1];
    }
    break;
```

For a diagonal matrix, we only need diagonal values.

So if `n = 3`, we only ask for 3 values.

Example input:

```
5 8 9
```

Storage:

```
A[0] = 5
A[1] = 8
A[2] = 9
```

This array represents the full matrix:

```
5 0 0
0 8 0
0 0 9
```

14.1 Create dry run

Suppose:

$n = 3$

The loop runs like this:

First loop

$i = 1$

Input:

5

Store:

$A[i - 1] = A[1 - 1] = A[0] = 5$

Second loop

$i = 2$

Input:

8

Store:

$A[i - 1] = A[2 - 1] = A[1] = 8$

Third loop

$i = 3$

Input:

9

Store:

```
A[i - 1] = A[3 - 1] = A[2] = 9
```

Final stored array:

```
A = [5, 8, 9]
```

15. Menu option 2: Get

The get option returns the value at a matrix position.

Code:

```
case 2:
    cout << "Enter row and column: ";
    cin >> i >> j;

    if (i == j) {
        cout << "Element is " << A[i - 1] << endl;
    } else {
        cout << "Element is 0" << endl;
    }
    break;
```

The program first asks for:

```
row and column
```

Then it checks:

```
i == j
```

If this is true, the position is diagonal.

So the program returns:

```
A[i - 1]
```

If this is false, the position is not diagonal.

So the program returns:

0

15.1 Get dry run for a diagonal position

Assume:

A = [5, 8, 9]

The user chooses option and enters:

2 2

So:

i = 2
j = 2

Check:

i == j
2 == 2

This is true.

Calculate index:

index = i - 1
index = 2 - 1
index = 1

Return:

A[1] = 8

Output:

```
Element is 8
```

15.2 Get dry run for a non-diagonal position

Assume:

```
A = [5, 8, 9]
```

The user enters:

```
1 3
```

So:

```
i = 1  
j = 3
```

Check:

```
i == j  
1 == 3
```

This is false.

So the program does not check the array.

It directly prints:

```
Element is 0
```

Why?

Because `M[1][3]` is a structural zero in a diagonal matrix.

16. Menu option 3: Set

The set option changes a value in the matrix.

Code:

```
case 3:
    cout << "Enter row, column, and new value: ";
    cin >> i >> j >> x;

    if (i == j) {
        A[i - 1] = x;
        cout << "Element updated.\n";
    } else {
        cout << "Cannot set non-diagonal position in a diagonal matrix.\n";
    }
    break;
```

The user enters:

```
row column value
```

The program checks whether the position is diagonal.

If:

```
i == j
```

then the value is stored.

If:

```
i != j
```

then the value is rejected.

This protects the diagonal matrix rule.

16.1 Set dry run for a diagonal position

Assume:

```
A = [5, 8, 9]
```

The user enters:

```
3 3 10
```

So:

```
i = 3  
j = 3  
x = 10
```

Check:

```
i == j  
3 == 3
```

This is true.

Calculate index:

```
index = i - 1  
index = 3 - 1  
index = 2
```

Store:

```
A[2] = 10
```

Now the array becomes:

```
A = [5, 8, 10]
```

The visible matrix becomes:

```
5 0 0
0 8 0
0 0 10
```

16.2 Set dry run for a non-diagonal position

Assume:

```
A = [5, 8, 9]
```

The user enters:

```
1 3 99
```

So:

```
i = 1
j = 3
x = 99
```

Check:

```
i == j
1 == 3
```

This is false.

So the program does not store `99`.

Output:

```
Cannot set non-diagonal position in a diagonal matrix.
```

The array remains:

```
A = [5, 8, 9]
```

Why?

Because `M[1][3]` must stay zero in a diagonal matrix.

17. Menu option 4: Display

The display option prints the full matrix.

Code:

```
case 4:
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (i == j) {
                cout << A[i - 1] << " ";
            } else {
                cout << "0 ";
            }
        }
        cout << endl;
    }
    break;
```

The array stores only diagonal values.

But display must print the full square matrix.

So it uses two loops:

```
for (i = 1; i <= n; i++)
```

This loop controls rows.

```
for (j = 1; j <= n; j++)
```

This loop controls columns.

For every position:


```
if (i == j)
```

print the stored value.

Otherwise, print:

```
0
```

17.1 Display dry run

Assume:

```
n = 3  
A = [5, 8, 10]
```

The visible matrix should be:

```
5 0 0  
0 8 0  
0 0 10
```

Row 1

Position (1,1):

```
1 == 1
```

Print:

```
A[0] = 5
```

Position (1,2):

```
1 == 2
```

False. Print 0.

Position (1,3) :

```
1 == 3
```

False. Print 0.

Row 1 output:

```
5 0 0
```

Row 2

Position (2,1) is not diagonal, so print 0.

Position (2,2) is diagonal, so print:

```
A[1] = 8
```

Position (2,3) is not diagonal, so print 0.

Row 2 output:

```
0 8 0
```

Row 3

Position (3,1) is not diagonal, so print 0.

Position (3,2) is not diagonal, so print 0.

Position (3,3) is diagonal, so print:

```
A[2] = 10
```

Row 3 output:

```
0 0 10
```

Full output:

```
5 0 0
0 8 0
0 0 10
```

18. Full program dry run

Suppose the user enters:

```
Enter matrix dimension: 3
```

The program creates:

```
A = [0, 0, 0]
```

because of:

```
A = new int[n]{};
```

Step 1: User chooses Create

Menu choice:

```
1
```

User enters diagonal elements:

```
5 8 9
```

The array becomes:

```
A = [5, 8, 9]
```

This represents:

```
5 0 0
0 8 0
0 0 9
```

Step 2: User chooses Get

Menu choice:

```
2
```

User enters:

```
2 2
```

Check:

```
2 == 2
```

True.

Index:

```
2 - 1 = 1
```

Return:

```
A[1] = 8
```

Output:

```
Element is 8
```

Step 3: User chooses Set

Menu choice:

3

User enters:

3 3 10

Check:

3 == 3

True.

Index:

3 - 1 = 2

Store:

A[2] = 10

The array becomes:

A = [5, 8, 10]

Step 4: User chooses Display

Menu choice:

4

The program prints:

```
5 0 0
0 8 0
0 0 10
```

Step 5: User chooses Exit

Menu choice:

```
0
```

The program prints:

```
Exiting program.
```

Then it releases memory:

```
delete[] A;
```

The program ends.

19. C-style dynamic allocation version

In C++, we use:

```
A = new int[n]{};
```

In C, we would use:

```
A = (int *)malloc(n * sizeof(int));
```

For a diagonal matrix, the size is `n` integers.

C memory rule:

```
malloc() must be matched with free()
```

Example:

```
A = (int *)malloc(n * sizeof(int));  
free(A);
```

C++ memory rule:

```
new[] must be matched with delete[]
```

Example:

```
A = new int[n]{};  
delete[] A;
```

20. Why display is slower than get and set

The `get` operation checks one position.

Example:

```
get M[2][2]
```

It only checks:

```
i == j
```

Then it returns one value.

So `get` is constant time.

The `set` operation also checks one position and possibly updates one array cell.

So `set` is also constant time.

But `display` must print every visible position.

For an `n x n` matrix, that means:

$n * n$

positions.

So display takes more time.

21. Complexity of the diagonal menu program

Operation	Time complexity	Reason
Create	$O(N)$	Reads N diagonal values
Get	$O(1)$	Checks one position and returns one value
Set	$O(1)$	Checks one position and updates one value
Display	$O(N^2)$	Prints all $N \times N$ visible positions
Storage	$O(N)$	Stores only N diagonal values

Important point:

The display function is $O(N^2)$ even though storage is only $O(N)$.

Why?

Because storage is compact, but display must show the full matrix.

22. Adapting the menu program to lower triangular row-major storage

This is one of the most important ideas in Lesson 13.

The menu structure does not need to change much.

The menu can still be:

1. Create
2. Get
3. Set


```
4. Display
0. Exit
```

The loop can still be:

```
do {
    // menu
} while (choice != 0);
```

The `switch` can still have the same cases.

What changes is the matrix logic.

To change the program from diagonal matrix to lower triangular row-major matrix, change these three things:

1. Array size
2. Stored condition
3. Index formula

22.1 Change 1: Array size

For a diagonal matrix, storage size is:

```
n
```

So allocation is:

```
A = new int[n]{};
```

For a lower triangular matrix, storage size is:

```
n * (n + 1) / 2
```

So allocation becomes:

```
A = new int[n * (n + 1) / 2]{};
```

Why?

A lower triangular matrix stores:

```
Row 1: 1 value  
Row 2: 2 values  
Row 3: 3 values  
...  
Row n: n values
```

Total:

$$1 + 2 + 3 + \dots + n = n * (n + 1) / 2$$

22.2 Change 2: Stored condition

For a diagonal matrix, the stored condition is:

```
i == j
```

For a lower triangular matrix, the stored condition is:

```
i >= j
```

Why?

Lower triangular matrices store values on and below the main diagonal.

Examples:

```
M[1][1] stored  
M[2][1] stored  
M[2][2] stored  
M[3][1] stored  
M[3][2] stored  
M[3][3] stored
```

These all follow:

$$i \geq j$$

Positions above the diagonal follow:

$$i < j$$

Those are structural zeros.

22.3 Change 3: Index formula

For a diagonal matrix, the index formula is:

$$\text{index} = i - 1$$

For lower triangular row-major storage, the index formula is:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

This formula means:

Skip previous rows + move inside current row

The first part:

$$i * (i - 1) / 2$$

skips all stored values before row i .

The second part:

$$j - 1$$

moves to the correct column inside the current row.

22.4 Comparison table

Program part	Diagonal matrix	Lower triangular row-major matrix
Stored condition	<code>i == j</code>	<code>i >= j</code>
Zero condition	<code>i != j</code>	<code>i < j</code>
Array size	<code>n</code>	<code>n * (n + 1) / 2</code>
Index formula	<code>i - 1</code>	<code>i * (i - 1) / 2 + (j - 1)</code>
Get outside stored region	return <code>0</code>	return <code>0</code>
Display	print values or fake zeros	print values or fake zeros

23. Lower triangular version of get logic

Diagonal get logic:

```
if (i == j) {  
    cout << A[i - 1];  
} else {  
    cout << 0;  
}
```

Lower triangular row-major get logic:

```
if (i >= j) {  
    int index = i * (i - 1) / 2 + (j - 1);  
    cout << A[index];  
} else {  
    cout << 0;  
}
```

Only the condition and formula changed.

The idea is the same.

24. Lower triangular version of set logic

Diagonal set logic:

```
if (i == j) {  
    A[i - 1] = x;  
}
```

Lower triangular row-major set logic:

```
if (i >= j) {  
    int index = i * (i - 1) / 2 + (j - 1);  
    A[index] = x;  
}
```

If:

```
i < j
```

then the position is above the main diagonal.

So the program should not store the value.

25. Lower triangular version of display logic

Diagonal display condition:

```
if (i == j)
```

Lower triangular display condition:

```
if (i >= j)
```

Diagonal display value:

```
A[i - 1]
```

Lower triangular display value:

```
A[i * (i - 1) / 2 + (j - 1)]
```

So the display loop becomes:

```
for (i = 1; i <= n; i++) {
    for (j = 1; j <= n; j++) {
        if (i >= j) {
            int index = i * (i - 1) / 2 + (j - 1);
            cout << A[index] << " ";
        } else {
            cout << "0 ";
        }
    }
    cout << endl;
}
```

26. The real pattern of Lesson 13

The most important lesson is this:

The menu structure stays mostly the same.
The matrix formula changes.

For any special matrix, we usually need:

1. Storage size
2. Stored condition
3. Index formula
4. Get rule
5. Set rule
6. Display rule

Once those are known, we can plug them into the same menu structure.

27. Common beginner mistakes

Mistake 1: Allocating `n * n` memory for a special matrix

Wrong for diagonal matrix:

```
A = new int[n * n];
```

This wastes memory.

Correct:

```
A = new int[n]{};
```

A diagonal matrix stores only `n` values.

Mistake 2: Forgetting that matrix positions are 1-based

Matrix position:

```
M[1][1]
```

Array index:

```
A[0]
```

So the formula is:

```
i - 1
```

not:

```
i
```

Mistake 3: Storing non-diagonal values in a diagonal matrix

Wrong idea:

```
A[someIndex] = 99; // for M[1][3]
```

This breaks the diagonal matrix rule.

Correct behavior:

```
if (i == j) {  
    A[i - 1] = x;  
} else {  
    // do not store  
}
```

Mistake 4: Forgetting `break` in switch-case

Wrong:

```
case 1:  
    // create  
case 2:  
    // get
```

Without `break`, the program may continue into the next case.

Correct:

```
case 1:  
    // create  
    break;
```

Mistake 5: Forgetting to release memory

If we use:

```
A = new int[n]{};
```

we should later write:

```
delete[] A;
```

Otherwise, the program may leak memory.

Mistake 6: Using `delete` instead of `delete[]`

Wrong:

```
delete A;
```

Correct:

```
delete[] A;
```

Use `delete[]` because `A` points to an array.

28. Key formulas from Lesson 13

Diagonal matrix

Stored condition:

```
i == j
```

Zero condition:

```
i != j
```

Storage size:

```
n
```

Index formula:

```
index = i - 1
```

Lower triangular row-major matrix

Stored condition:

```
i >= j
```

Zero condition:

```
i < j
```

Storage size:

```
n * (n + 1) / 2
```

Index formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

29. Final summary

Lesson 13 teaches how to build a menu-driven matrix program.

The main idea is not a new matrix formula.

The main idea is organizing matrix operations into a repeated menu.

The program repeatedly asks the user to choose an operation:

```
Create  
Get  
Set  
Display  
Exit
```

For the diagonal matrix version:

```
condition = i == j  
index = i - 1  
array size = n
```

The program stores only diagonal values and prints fake zeros for non-diagonal positions.

For the lower triangular version:

```
condition = i >= j
index = i * (i - 1) / 2 + (j - 1)
array size = n * (n + 1) / 2
```

The important reusable idea is:

```
Keep the menu.
Change the condition.
Change the array size.
Change the index formula.
```

That is the core pattern behind menu-driven programs for special matrices.

30. What you should be able to explain after Lesson 13

After this lesson, you should be able to explain these points:

1. What a menu-driven program is.
2. Why a `do-while` loop is useful for menus.
3. Why `switch-case` is useful for handling menu choices.
4. How a diagonal matrix is stored in a 1D array.
5. Why diagonal matrix storage uses only `n` values.
6. Why non-diagonal values are structural zeros.
7. How the create option fills the stored array.
8. How the get option returns a stored value or zero.
9. How the set option updates only valid stored positions.
10. How the display option prints the full visible matrix.
11. Why display is `O(N2)` even though storage is `O(N)`.
12. How to adapt the same menu program to a lower triangular matrix.
13. Which three things change when switching matrix type: array size, stored condition, and index formula.